

REST API Specification: AI Virtual Try-On (VTO)

This document defines the API endpoints required to power the VTO platform, focusing on a clean separation between user actions, merchant management, and the AI processing pipeline.

Base URL

https://api.vto-startup.com/v1

1. Authentication & User Management

Endpoint	Method	Input (JSON)	Output (JSON)
/auth/register	POST	email , password , role	user_id , token
/auth/login	POST	email , password	user_id , token , role

2. Human Image Management (Customer)

Endpoint	Method	Input	Output (JSON)
/user/photos	POST	multipart/form-data (file)	photo_id , url
/user/photos	GET	Header: Auth Token	Array of {photo_id, url}
/user/photos/{id}	DELETE	Path Variable: id	status: "deleted"

3. Product & Catalog (Merchant/Public)

Endpoint	Method	Input	Output (JSON)
/products	GET	Query: category , limit	Array of products
/merchant/products	POST	multipart/form-data (garment file, name, price)	product_id , status
/merchant/products/{id}	PATCH	price , stock_status	updated_product

4. AI Try-On Pipeline (The Core)

Note: AI generation is asynchronous. The `POST` starts the task, and the user polls or waits for a socket.

Endpoint	Method	Input (JSON)	Output (JSON)
/tryon/generate	POST	photo_id , product_id	task_id , estimated_wait
/tryon/status/{task_id}	GET	Path Variable: task_id	status: "processing/completed" , result_url
/tryon/history	GET	Header: Auth Token	Array of previous results

5. Merchant Dashboard & Analytics

Endpoint	Method	Input	Output (JSON)
----------	--------	-------	---------------

<code>/analytics/summary</code>	GET	Header: Merchant Token	<code>total_tryons</code> , <code>conversion_rate</code> , <code>top_items</code>
<code>/analytics/activity</code>	GET	Query: <code>start_date</code> , <code>end_date</code>	Array of { <code>date</code> , <code>count</code> }

Technical Implementation Details

A. Asynchronous AI Handling

Because the AI pipeline (Preprocessing → Segmentation → Warping → Generation) can take 5–15 seconds, the `/tryon/generate` endpoint should not keep the HTTP connection open.

1. **Request:** Client sends `photo_id` and `product_id`.
2. **Queue:** Backend adds the request to a message broker (e.g., Redis or RabbitMQ).
3. **Worker:** A GPU-enabled worker picks up the job and runs the pipeline.
4. **Notification:** Once finished, the worker updates the database status. The client gets the result via WebSockets or Polling the `/status` endpoint.

B. Image Storage

- **Input:** Original images should be stored in an S3 bucket under `/uploads/raw/`.
- **Output:** Generated results should be stored under `/results/processed/`.
- **CDN:** Use a Content Delivery Network (like CloudFront) to serve these images to users with low latency.

C. Scalability

- **Statelessness:** Ensure the API server is stateless (using JWT for auth) so you can spin up multiple instances.

- **Rate Limiting:** Implement rate limiting on the `/tryon/generate` endpoint to prevent GPU cost spikes from bot attacks.